

Customer Support Operations Platform

Frontend Internship Project — React and Angular

1. Business Context

Customers currently report problems using phone calls, email, and messaging applications.

Customers do not have one place to follow their requests, and the support team does not have one operational view of ownership, urgency, progress, and communication.

The company wants two connected application experiences using the same business data:

- a customer portal,
- a support-team workspace.

2. Business Goals

- Give customers one place to create and follow support requests.
- Give support employees one place to manage operational work.
- Make request ownership, urgency, and progress clear.
- Keep customer communication connected to the request.
- Keep internal support communication private.
- Provide a professional experience across desktop and smaller screens.
- Keep protected information protected.
- Support growing support-request data.

3. Applications

Customer Portal — React

Used by customers to create requests, follow progress, communicate with support, and review resolution.

Support Workspace — Angular

Used by support agents and managers to find work, manage ownership, communicate, update progress, and review workload.

4. Shared Data Source

Both applications must use the same underlying business data.

The intern is responsible for selecting and preparing an appropriate shared backend/data service.

A managed service such as Supabase is suitable.

5. General Business Constraints

- Customers must only access their own data.
- Internal support notes must never be exposed to customers.
- Security must not depend only on hiding user-interface actions.
- Invalid actions must be handled safely.
- Important screens should clearly represent loading, empty, validation, failure, and success states.
- The experience should remain practical as the amount of request data grows.

6. General Delivery Expectations

- Source code for both applications in a Git repository.
- Shared data-source setup instructions.
- Clear local run instructions.
- Safe configuration examples.
- Test users or documented user-creation steps.
- Assumptions, decisions, known limitations, and incomplete items.
- Appropriate automated tests.
- Meaningful Git commit history.